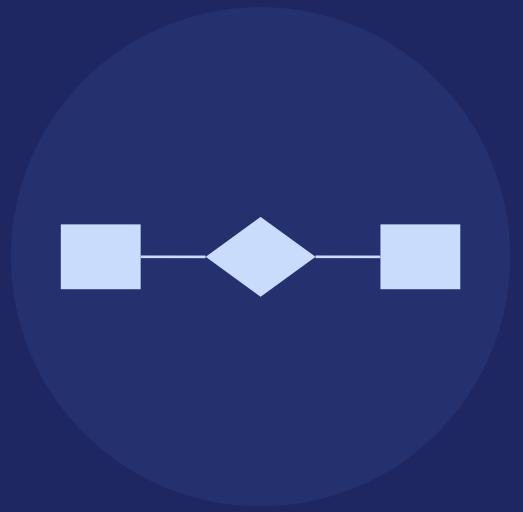
MODELAGEM DE DADOS · AULA

Modelagem Física com SQLite

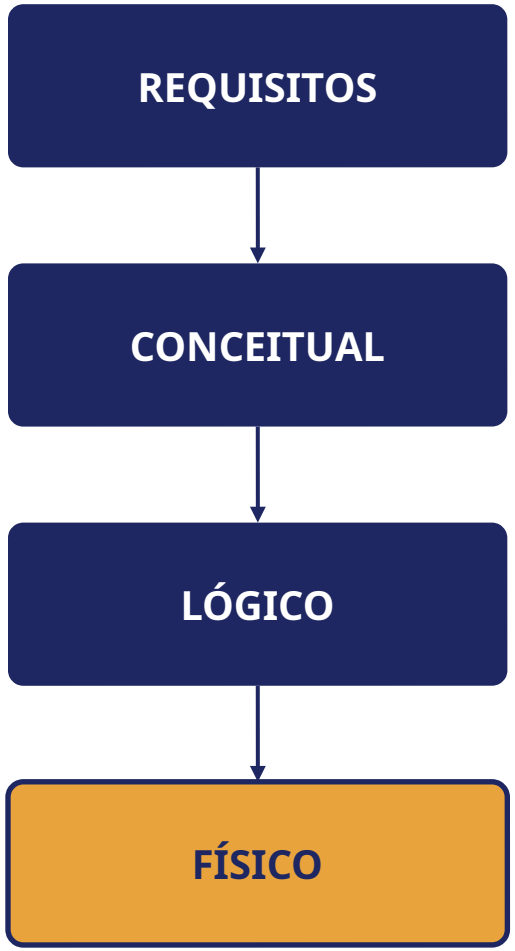
Catálogo de Filmes: do modelo lógico ao banco de dados real

SQL, tipos de dados, restrições e o DB Browser for SQLite (DB4S)



O ÚLTIMO DEGRAU

Onde Estamos



Documentação baseada nas entrevistas com o cliente.

O MER: entidades, atributos e relacionamentos.

As tabelas, colunas e chaves — já resolvemos isso.

Hoje: o banco vira um arquivo .db de verdade, com SQLite.

A ESCOLHA DA AULA

Por que SQLite?

Sem servidor

O banco é um único arquivo .db — nada para instalar, nenhum processo rodando.

Zero configuração

Sem usuário, senha ou porta de rede para configurar antes de começar.

SQL padrão

Usa a mesma linguagem SQL de MySQL, PostgreSQL e outros SGBDs.

Ótimo para aprender

Foco no conteúdo de modelagem, não na infraestrutura do banco.

É o mesmo motor de banco de dados usado dentro do seu navegador e do seu celular.

A LINGUAGEM DO BANCO DE DADOS

Introdução ao SQL

- SQL (Structured Query Language) é a linguagem usada para criar e manipular bancos de dados relacionais
- **Ela se divide em duas partes:**
  - DDL (Data Definition Language) — define a estrutura: tabelas, colunas, restrições
  - DML (Data Manipulation Language) — manipula os dados: inserir, consultar, alterar, apagar

Comando	Categoria	O que faz
`CREATE TABLE`	DDL	Define a estrutura de uma tabela
`DROP TABLE`	DDL	Remove uma tabela e todos os seus dados
`INSERT INTO`	DML	Insere registros (linhas) na tabela
`SELECT`	DML	Consulta registros já existentes

Essas quatro instruções são o suficiente para modelar e testar o banco inteiro de hoje.

1º COMANDO — DDL

CREATE TABLE

```sql
CREATE TABLE nome_tabela (
  coluna1 TIPO PRIMARY KEY,
  coluna2 TIPO NOT NULL,
  coluna3 TIPO
);
```

- Cria uma tabela nova, com nome e lista de colunas
- Cada coluna tem um tipo de dado (veremos os tipos do SQLite já já)
- **PRIMARY KEY identifica cada linha de forma única — é a chave primária que já conhecemos do modelo lógico**

2º COMANDO — DDL

DROP TABLE

```
DROP TABLE nome_tabela;

DROP TABLE IF EXISTS nome_tabela;
```

Cuidado: DROP TABLE apaga a tabela inteira e todos os seus dados, sem pedir confirmação. Não existe "desfazer".

- IF EXISTS evita erro quando a tabela ainda não existe — útil para rodar o script várias vezes durante a aula

3º COMANDO — DML

INSERT INTO

```
INSERT INTO tabela (coluna1, coluna2)
VALUES (valor1, valor2);

INSERT INTO tabela
VALUES (valor1, valor2, valor3);
```

- **Com a lista de colunas:**
  - mais seguro — não depende da ordem das colunas da tabela
- **Sem a lista de colunas:**
  - mais curto, mas exige informar todos os valores, na ordem exata em que as colunas foram criadas

4º COMANDO — DML

SELECT

```
SELECT coluna1, coluna2
FROM tabela
WHERE condicao;


SELECT * FROM tabela;
```

- * seleciona todas as colunas da tabela
- WHERE filtra quais linhas aparecem no resultado (é opcional)
- **É a instrução que vamos usar o tempo todo para conferir se os dados foram inseridos corretamente**

CATÁLOGO DE FILMES

O Cenário

Uma plataforma online para catalogar filmes, com elenco, direção, avaliações dos usuários e listas de favoritos.

Filme

Estúdio

Classificação

País

Gênero

Profissional

Usuário

Comentário

O modelo lógico completo (com as chaves e os relacionamentos) já foi resolvido nas aulas anteriores.

O QUE MUDA

Do Lógico ao Físico

- **Tipos de dados concretos**
  - “texto” vira TEXT, “número” vira INTEGER ou REAL
- **Nomes técnicos**
  - sem espaços ou acentos — cod_filme, ano_lancamento
- **Chaves estrangeiras ligadas explicitamente**
  - PRAGMA foreign_keys = ON; — no SQLite, vêm desligadas por padrão!
- **Restrições de integridade**
  - CHECK, UNIQUE, DEFAULT — o modelo lógico não define isso, o físico sim

É aqui que o desenho vira, de fato, um banco de dados que pode ser quebrado — ou protegido.

SÓ QUATRO TIPOS

Tipos de Dados no SQLite

Tipo	Usado para	Exemplo no cenário
`INTEGER`	números inteiros	`cod_filme, ano_lancamento, duracao`
`TEXT`	textos e strings	`titulo, nome, email, sinopse`
`REAL`	números com casas decimais	*(não usamos no cenário de hoje)*
`BLOB`	dados binários	*(ex.: pôster do filme em imagem)*

Atenção: o SQLite tem tipagem dinâmica (type affinity) — ele aceita guardar um texto numa coluna INTEGER, por exemplo. Vale ser disciplinado ao definir os tipos mesmo assim.

AS MAIS SIMPLES PRIMEIRO

Tabela: País, Classificação, Gênero

pais

```
CREATE TABLE pais (
  cod_pais INTEGER
    PRIMARY KEY,
  nome TEXT NOT NULL
);
```

classificacao

```
CREATE TABLE
  classificacao (
  cod_class INTEGER
    PRIMARY KEY,
  descricao TEXT
    NOT NULL
);
```

genero

```
CREATE TABLE genero (
  cod_genero INTEGER
    PRIMARY KEY,
  nome TEXT NOT NULL
);
```

- As três seguem a mesma estrutura: um código como PRIMARY KEY e um NOT NULL
- Nenhuma tem FOREIGN KEY — são independentes de qualquer outra tabela

INSERT + SELECT

Primeiros Dados

```sql
INSERT INTO pais (nome)
VALUES ('Brasil');

INSERT INTO pais (nome)
VALUES ('Estados Unidos');

SELECT * FROM pais;
```

Resultado:

cod_pais	nome
1	Brasil
2	Estados Unidos

- **Repare: não informamos cod_pais no INSERT**
  - Quando a PRIMARY KEY é INTEGER, o SQLite gera o valor automaticamente (rowid / autoincremento implícito)

MESMO PADRÃO, UM PASSO ANTES DO FILME

Tabela: Estúdio

```sql
CREATE TABLE estudio (
  cod_estudio INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  ano_fundacao INTEGER
);
```

- Ainda sem FOREIGN KEY — Estúdio também é independente
- **A partir da próxima tabela, isso muda: Filme vai referenciar Estúdio, Classificação e País**

FOREIGN KEY, CHECK E DEFAULT

Tabela: Filme

```sql
PRAGMA foreign_keys = ON;

CREATE TABLE filme (
  cod_filme INTEGER PRIMARY KEY,
  titulo TEXT NOT NULL,
  ano_lancamento INTEGER CHECK (ano_lancamento > 1888),
  duracao INTEGER CHECK (duracao > 0),
  sinopse TEXT,
  cod_estudio INTEGER NOT NULL,
  cod_class INTEGER NOT NULL,
  cod_pais INTEGER NOT NULL,
  FOREIGN KEY (cod_estudio) REFERENCES estudio (cod_estudio),
  FOREIGN KEY (cod_class) REFERENCES classificacao (cod_class),
  FOREIGN KEY (cod_pais) REFERENCES pais (cod_pais)
);
```

PRAGMA foreign_keys = ON precisa rodar a cada conexão — o SQLite não guarda essa opção no arquivo .db.

O QUE ACONTECE SE EU TENTAR BURLAR?

Testando a Integridade

```sql
-- estudio com cod_estudio = 99 nao existe
INSERT INTO filme
  (titulo, ano_lancamento, duracao, cod_estudio, cod_class, cod_pais)
VALUES
  ('Filme Fantasma', 2024, 100, 99, 1, 1);
```

```
sqlite> Error: FOREIGN KEY constraint failed
```

- Com PRAGMA foreign_keys = ON, o SQLite recusa o INSERT — a integridade é garantida pelo próprio banco
- **Sem esse PRAGMA, o INSERT passaria normalmente, e o banco ficaria com um dado “fantasma”**

ATORES E DIRETORES, UMA SÓ ENTIDADE

Tabela: Profissional

```sql
CREATE TABLE profissional (
  cod_prof INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  nascimento TEXT,
  cod_pais INTEGER,
  FOREIGN KEY (cod_pais) REFERENCES pais (cod_pais)
);
```

- nascimento é TEXT porque o SQLite não tem um tipo DATE nativo — datas costumam ser guardadas como texto no formato 'AAAA-MM-DD'
- Relembrando: usamos Profissional (e não Ator/Diretor separados) para não duplicar dados de quem atua e dirige ao mesmo tempo

PRIMEIRA TABELA N:N

Tabela: Gênero × Filme

```
CREATE TABLE filme_genero (
  cod_filme INTEGER,
  cod_genero INTEGER,
  PRIMARY KEY (cod_filme, cod_genero),
  FOREIGN KEY (cod_filme) REFERENCES filme (cod_filme),
  FOREIGN KEY (cod_genero) REFERENCES genero (cod_genero)
);
```

- **A PRIMARY KEY agora é composta — a combinação das duas colunas**
- Garante que o mesmo filme não seja associado duas vezes ao mesmo gênero
- Tem duas FOREIGN KEY, uma para cada entidade envolvida — é a estratégia “tabela própria” que vimos no modelo lógico

N:N COM ATRIBUTO × N:N SEM ATRIBUTO

Tabela: Atuação e Direção

atuacao

```
CREATE TABLE atuacao (
  cod_filme INTEGER,
  cod_prof INTEGER,
  personagem TEXT,
  PRIMARY KEY
    (cod_filme, cod_prof),
  FOREIGN KEY (cod_filme)
    REFERENCES filme (cod_filme),
  FOREIGN KEY (cod_prof)
    REFERENCES
    profissional (cod_prof)
);
```

direcao

```
CREATE TABLE direcao (
  cod_filme INTEGER,
  cod_prof INTEGER,
  PRIMARY KEY
    (cod_filme, cod_prof),
  FOREIGN KEY (cod_filme)
    REFERENCES filme (cod_filme),
  FOREIGN KEY (cod_prof)
    REFERENCES
    profissional (cod_prof)
);
```

personagem só faz sentido na atuação — por isso direcao não tem essa coluna extra.

UNIQUE E DEFAULT

Tabela: Usuário

```
CREATE TABLE usuario (
  cod_usuario INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  data_cadastro TEXT DEFAULT CURRENT_TIMESTAMP
);
```

- UNIQUE impede dois usuários com o mesmo e-mail
- DEFAULT CURRENT_TIMESTAMP preenche a data automaticamente, se o INSERT não informar um valor

RELACIONAMENTO × ENTIDADE PRÓPRIA

Tabela: Favorito e Comentário

favorito

```sql
CREATE TABLE favorito (
  cod_usuario INTEGER,
  cod_filme INTEGER,
  data_adicao TEXT
    DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY
    (cod_usuario, cod_filme),
  FOREIGN KEY (cod_usuario)
    REFERENCES usuario (...),
  FOREIGN KEY (cod_filme)
    REFERENCES filme (...)
);
```

comentario

```sql
CREATE TABLE comentario (
  cod_coment INTEGER
    PRIMARY KEY,
  texto TEXT,
  nota INTEGER NOT NULL
    CHECK (nota BETWEEN 1 AND 5),
  data_public TEXT
    DEFAULT CURRENT_TIMESTAMP,
  cod_usuario INTEGER NOT NULL,
  cod_filme INTEGER NOT NULL
);
```

Favorito é só um relacionamento com atributo; Comentário virou entidade própria — um usuário pode comentar o mesmo filme mais de uma vez.

COLOCANDO TUDO PARA FUNCIONAR JUNTO

Consultas Finais

```sql
-- filmes com o nome do estudio
SELECT f.titulo, e.nome AS estudio
FROM filme f JOIN estudio e ON f.cod_estudio = e.cod_estudio;
```

```sql
-- nota media de cada filme
SELECT cod_filme, AVG(nota) AS media
FROM comentario GROUP BY cod_filme;
```

```sql
-- profissionais que atuam E dirigem
SELECT p.nome FROM profissional p
WHERE p.cod_prof IN (SELECT cod_prof FROM atuacao)
  AND p.cod_prof IN (SELECT cod_prof FROM direcao);
```

Essa última consulta é a prova de que a decisão de usar Profissional (em vez de Ator e Diretor separados) valeu a pena.

O que vocês construíram hoje

- Um arquivo .db real, aberto no DB Browser for SQLite (DB4S)
- 8 tabelas, com PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE e DEFAULT
- Testaram a integridade na prática — e viram o banco recusar um dado inválido
- Consultas que unem várias tabelas para responder perguntas reais do catálogo

---

Do MER ao arquivo .db: modelagem conceitual, lógica e física, de ponta a ponta.